Shorter correctness proofs and a combined 215 state machine

Research report

21 October 2026

Abstract

The lowest-risk route to shorter certified proofs is to retain the already certified K215 machine and remove duplicated proof infrastructure. A generic symmetry theorem for offset bisimulation can replace 2,240 reversed checks in 16 Lean files. An independently audited compact representation reduces the Rocq certificate data from 1,499,170 to 190,411 bytes without changing a single decoded row. The supplied Lean and Rocq refactor prototypes are *uncompiled*, not new kernel-certified proofs. For a fresh formalization, the exhaustive zero-debt scheduler of ZC215 offers a simple injection argument; combining it with command-tag generalization gives GC–ZC215, a concrete 215-state construction with a shorter GEN block. Its binary runtime and blank startup are separately checked. No 214-state construction is established.

1 The concrete proof refactor

The pinned upstream snapshot is commit `c9dc79323429112c698100f3226a6e4d56c8b782`. Independent data inspection establishes that all 2,240 reverse entries in the Lean source certificate are exact mirrors of the forward entries. The corresponding Rocq certificate has exactly the same rows. All present offsets happen to be zero, but the proposed theorem handles arbitrary integer offsets.

For a relation $S(A, B, z)$ on source paths and register offsets, define

$$S^{\mathrm{rev}}(B, A, z) = S(A, B, -z).$$

The essential proof obligation is $\delta(-z, i) = -\delta(z, i)$. Swap the two branch witnesses, negate the successor offset, and reuse the same successor relation. This converts branch correctness, entry correctness and finally offset bisimulation into their reverse forms. Applying the existing forward halting simulation twice gives

$$\mathrm{OBisim}(p, q, S) \quad \Longrightarrow \quad \mathrm{Halts}(p, R) \iff \mathrm{Halts}(q, R).$$

Thus a forward certificate need not be checked again as a separate reversed certificate. This removes duplication, not a direction of the final theorem.

The included 63-line `OffsetBisimSymmetry.lean` is a concrete proposed integration. It contains no added axiom, admission or `native_decide`; it ends with an axiom-printing request. However, absence of these constructs is not a substitute for compilation. The project imports, elaboration, tactic applications and final dependency closure still have to pass the pinned Lean kernel. No Lean or Rocq compiler was available in the research environment.

1.1 Size and timing evidence

The historical build log attributes 1,118.7394 seconds to the 16 backward-check files, out of 4,485.4087 seconds summed over 204 dependency compilations: approximately 24.94%. These are *historical sums*

of individual compilation costs, not elapsed wall time and not a measured speedup of a rebuilt refactor. Parallelism, cache effects and the generic lemma's new cost must be measured in a clean comparison.

The Rocq data refactor represents each Boolean path by a positive binary integer with a leading sentinel bit. Recursive decoding strips that sentinel while preserving every leading and trailing zero. Mapping this decoder over the compact tuples reconstructs all 2,240 original rows exactly; both successor indices and signed offsets are unchanged. The textual size falls by about 87.30%. This is a data-size result, not a demonstrated kernel-time reduction. A computational equality lemma and the existing checker soundness theorem are the right integration boundary.

1.2 A second stage for the zero offsets

Ordinary `RMBisim` already has symmetry and a bidirectional halting theorem. Because all 2,240 offsets are zero, a four-field natural-number certificate could replace the five-field offset certificate. There is an important decoding trap: the ordinary checker uses `decodeAux 128`, whereas the offset checker uses fuel 512. Exactly 622 pairs have a path longer than 128 bits; the maximum is 176. Blindly dropping the offset fields changes these pointers and is invalid.

Parameterize the ordinary checker uniformly by its path decoder and instantiate it with the same 512-fuel decoder. Preserve certificate-tree cuts and successor indices, prove lookup commutes with field erasure, require an explicit all-offsets-zero guard, and translate offset-zero integer counts to ordinary natural counts. Then prove entry-check transport and apply the ordinary bidirectional theorem. This is a precise architectural plan, not a compiled refactor or measured additional speedup. Retaining the existing 128-fuel default avoids changing older clients.

2 How to reorganize the certified proof

Keep numerical-calculus soundness and completeness independent of each machine. In particular, preserve raw-natural codes and aliases: the executed enumerator need not generate only decodable formulas. Parameterize the generalization output of AX by a binder selector $\beta(U, C)$, with the current and command-tag choices as instances, instead of rebuilding the whole calculus.

Separate the following interfaces:

1. Command soundness preserves theorem-valued stores. Strong command completeness instead permits arbitrary incoming stores and an earlier successful halt. These are different statements.

2. Parsing proves round trips, field arity and strictly decreasing positive continuations. A generic scheduler theorem then supplies exhaustive coverage or cutoff-plus-cofinal coverage.

3. A generic runtime checker has one soundness theorem. Machine tables, source lists and certificates are untrusted data; prove their exact interpretation and accept them only through the checker.

4. A startup theorem identifies the complete finite tape and its zero exterior, true start state, dispatcher, source ordinal, registers, gap and spare conditions. A plausible register tuple is insufficient.

5. One endpoint composes startup, positive-duration bidirectional halting simulation, source calibration and numerical-calculus consistency calibration. Keep an external equivalence distinct from a theorem internal to PA.

For certified K215, the numerical initial stack is zero, so all its decoded heads are truth. Its soundness proof can use a theorem-valued-store invariant directly; do not route that instance through protected-tail resource arithmetic. Refactoring this existing endpoint is likely less work than certifying a different table from scratch.

3 An alternative scheduler with a short safety proof

ZC215 enumerates every raw candidate $0, 1, 2, \ldots$ using four commands DROP, NIP, MP and AX. DROP has tag 0 and NIP tag 1, with the same field arity. For a NIP occurrence (n, j), replace just that command by DROP and keep its position j. Strict coordinate monotonicity of the nested pairing code makes the new candidate m strictly smaller than n. Raising that DROP back to NIP recovers the original word, so $(n, j) \mapsto (m, j)$ is injective.

Every NIP in an executed finite prefix therefore has a distinct DROP in an earlier completed candidate. Immediately before a NIP, include that imminent pop among the sources: there are at least one more earlier push than past pops. No opaque stack tail is exposed. MP and AX need only be nonnegative contributors; no AX batch size, density estimate, reserve or numerical cutoff enters the argument.

This proof requires unique parsing, strict code monotonicity, equal DROP/NIP arity, exhaustive increasing enumeration beginning at zero, and unchanged stack history between candidates. It does not justify an arbitrary skipped-candidate schedule or a positive initial cutoff. A logical list of safe heads above an arbitrary tail handles numerical stack aliases without a false magnitude invariant.

The supplied 107-line `PrefixCredit.v` proposes reusable finite-list lemmas for an injective map, prefix credit and strict pre-pop credit. It uses only standard-library imports and explicit hypotheses, with no admission or added axiom. It is **uncompiled**. The machine-specific codec and runtime hypotheses remain separate obligations. An owned finite regression checks 100,000 candidates, 19,566 NIP images and 49,275 DROP commands; minimum prefix balance is zero. That finite test supports implementation debugging, not the all-natural theorem.

3.1 Which candidate is actually easier

K215 remains the least-risk choice for shortening an existing certified endpoint. ZC215 is a plausible easier alternative to ST215 or FT215 for a fresh proof: its true-root startup and occurrence injection avoid partial-AX startup matching and debt bounds. FT215's exact debt seven is useful research, but not a simpler proof. GC215 saves a local generalization block while retaining ST's other obligations. No measured complete Lean/Rocq proof-size comparison of these alternatives exists here.

4 The combined GC–ZC215 construction

The new combination retains ZC's four-command scheduler and changes AX's generalization head from $Q_U(\text{old } H)$ to $Q_C(\text{old } H)$, using the command's own tag C. All other axiom outputs, gates, dominated-row parameters and cleanup behavior remain unchanged. The zero-debt occurrence injection is unaffected because command arities, ordering and stack contributions are unchanged.

Writing B_X for the strong block returning $(X, w, 0, 0)$, the new GEN block is

$$B_X;\ \text{DROP};\ \text{NIP};\ \text{AX}(i; 0, 0);\ \text{NIP}^9;\ \text{DROP};\ \text{NIP};\ \text{NIP}.$$

DROP and NIP arrange $(0, w, 0, X)$. AX places $Q_i(X)$ in its ninth stack head and saved zero in the tenth. Nine NIPs load $Q_i(X)$ into cache; DROP makes it the top while saving the AX gate; two further

NIPs discard that gate and saved zero, restoring $(Q_i(X), w, 0, 0)$, or permitting an earlier successful halt.

The old parameter-loading block $J(i)$ disappears, but one must not silently reuse its old zero-gate proof. The new gate $G(0, X, i)$ can be enabled. It is nevertheless never falsity: it is either zero or implication-coded. The final NIP that sees it is therefore safe. This nonfalsity fact is an explicit additional proof obligation.

4.1 Physical realization and exact boundaries

The changed preserving parameter load initially allocates 216 states. A contextual decoder merge identifies new role 303 with role 129 and patches the read-one branch of 129 to write one, move left and enter 216. The marker proof is static: the only incoming edge of old role 129 writes one then moves right; the only incoming edge of old role 303 writes zero then moves right. Thus the predecessor cell identifies the origin on every entry. Four local traces, for both origin roles and read symbols, preserve the entire touched tape and endpoint. The portable checker reconstructs these traces and every incoming edge. Full universal runtime checks independently validate the resulting table's primitive contracts.

The 215-state table has 430 transitions and the source has 4,752 primitive sites. All 6,177 guarded cases pass the symbolic runtime checker, totaling 1,698,909 symbolic rules and covering all 430 rows. These quantified primitive contracts allow arbitrary natural registers, extra gap and nonempty finite spare lists; they are not just selected concrete simulations.

The corrected native artifact declares start 74 and halt 254; its SHA-256 is

`4122c18a347028ee6643428b70a8d5c81218d4937d672be3626981d7b50bf503`

The source SHA-256 is

`401bd1c9f08fcd63790b580f63ea942b1e4a1045cdd600bb1bd172fb06d01e8a`

The earlier runtime input mistakenly declared start 0 in its comment header. The corrected artifact changes that header alone, preserving all 430 transition rows byte for byte. The verifier checks this transport and independently replays the corrected table. Canonical relabeling uses start 0 and halt 254, with working labels 0 through 214; 254 is not counted among the 215 working states.

The valid native blank start is 74. After exactly 117,004 binary transitions, independent dense and sparse executions reach dispatcher 84 at head -4, source root code `000`, with logical registers

$$(N, T, W, A, D, H, X, Y, L) = (1, 0, 5, 0, 0, 0, 0, 0, 0).$$

The gap is 20 and some spare entries are nonzero, within the universal runtime's stated domain. The complete tape is matched, not merely the registers. The opaque stack tail 5 is admissible under the zero-debt injection. This direct root avoids a partial-AX source bridge. An earlier dispatcher encounter at 69,143 transitions is not the endpoint used for this claim.

Proof boundary. The combination is a concrete binary construction and a completed ordinary mathematical calibration with independently replayable runtime and startup checks. It is not a new Rocq- or Lean-certified endpoint. Nor does its shorter GEN block establish that its entire formal proof will be shorter than refactored K215. The exact-source audit and completed calibration are included in the accompanying evidence.

5 The 214 state question and correction

No checked construction with 214 or fewer states emerged. Removing a command block or reducing source DAG size does not imply removal of a binary state. Further attempts should co-design the proof and decoder: isolate a marker invariant or a stuttering-aware primitive refinement with a small reusable statement, then require exact startup and all-natural contracts before accepting any state deletion. The binder-choice Q_A trial reached 205 source nodes across 42 associations, but its three selected allocations failed primitive-prefix checks. A separate FT215 state-301 deletion failed the tested blank-start screens. Neither is a valid 214-state candidate. These bounded families and time-limited finite screens cannot prove impossibility.

The accompanying SB215 correction is independent of this result. Both newly inspected SB variants have a defective source return and must not be promoted as consistency machines. Their physical runtime receipts do not repair an incorrect source scheduler. The separately delivered FT215 construction is unaffected. The small correction archive supplies the exact reconciliation and check evidence rather than silently dropping the defect.

6 Reproduction and next formal milestone

The packet includes the exact native/canonical machine tables, source, universal runtime census, complete blank-root tape, owned portable checkers, proposed Lean/Rocq code, pinned upstream certificate inputs and comparison arguments. The verifier keeps every generated result outside the frozen payload. Run, using a fresh directory:

```
python -I -B verify.py --work /tmp/proof-check
python -I -B verify.py --work /tmp/proof-full --runtime
```

Assertions are part of the checker; optimized Python is rejected. The first command checks manifest integrity when present, exact certificate reconstruction, finite injection regressions, canonical relabeling and literal startup. The second additionally replays all 6,177 symbolic guards and compares their exact census. Neither command compiles Lean/Rocq or proves the numerical-calculus calibration.

The next formal milestone is narrow and testable: compile the symmetry lemma against the pinned Lean project, replace backward-check imports, print the endpoint's axioms and rebuild cleanly. Then prove compact Rocq decoding equality and replay the unchanged checker. Record source size, resulting proof objects, peak memory and both summed and wall-clock build time. Only after that comparison should the combined alternative be judged as a route to a shorter complete formal proof.